```
#basic imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from pybaseball import statcast
from pybaseball import pitching_stats
from pybaseball import batting_stats
from pybaseball import statcast_batter
from pybaseball import batting_stats_bref
from pybaseball import statcast_pitcher
from pybaseball.statcast_pitcher_spin import statcast_pitcher_spin
from pybaseball import playerid_lookup
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.cluster import DBSCAN
from sklearn.manifold import TSNE
from tensorflow import keras
from keras.preprocessing import sequence
from keras.models import Sequential
from keras.layers import Dense, Embedding
from keras.optimizers import Adam
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
import xgboost as xgb
from sklearn.metrics import accuracy_score, precision_score, recall_score,
mean_squared_error, mean_absolute_error
import sys
from sklearn.linear_model import LinearRegression, Lasso
from catboost import CatBoostRegressor
import seaborn as sns
plt.style.use('ggplot')

def spin_data(pitcher_id):
    spin_data = statcast_pitcher_spin('2019-07-01', '2019-09-30', player_id =
pitcher_id)
    datacluster = spin_data[['release_speed', 'release_spin_rate','pfx_x','pfx_z',
'spin_axis', 'effective_speed'  ]].dropna()
    datacluster1 = datacluster.dropna()
    datacluster1.mask(datacluster1.astype(object).eq('None')).dropna()
    datacluster1 = datacluster1.loc[datacluster1['spin_axis'] > 10]
    datacluster1

    scaler = StandardScaler()
    scaledcluster = scaler.fit_transform(datacluster1)
    scaledcluster = pd.DataFrame(scaledcluster)
    scaledcluster
```

```
    return scaledcluster

#function defined to allow input of both player names, then have their id returned
def get_player_id(pitcher_name, batter_name):

    #split pitcher's name, then put split name into pybaseball's id lookup function
    first_name_pitcher, last_name_pitcher = pitcher_name.split(' ',1)
    first_name_pitcher = first_name_pitcher
    last_name_pitcher = last_name_pitcher
    get_id_pitcher = playerid_lookup(last_name_pitcher,first_name_pitcher)

    #split batter's name, then put split name into pybaseball's id lookup function
    first_name_batter, last_name_batter = batter_name.split(' ',1)
    first_name_batter = first_name_batter
    last_name_batter = last_name_batter
    get_id_batter = playerid_lookup(last_name_batter,first_name_batter)

    #the above is return in a bataframe, so grabbing just what I need
    #the .to_string(index=False) is used to get rid of duplicate index
    pitcher_id = get_id_pitcher['key_mlbam'].to_string(index=False)
    batter_id = get_id_batter['key_mlbam'].to_string(index=False)


    return pitcher_id, batter_id


#defined function to use pitcher_id to grab data that will be use
def get_pitcher_data(pitcher_id):

    statcast_data_pitcher = statcast_pitcher('2018-07-01', '2022-10-31', player_id =
pitcher_id)
    #take data and select desired columns
    pitcher_data = statcast_data_pitcher[['pitch_type', 'description', 'stand',
'p_throws', 'on_3b','on_2b','on_1b', 'zone', 'pitcher', 'balls', 'inning','strikes',
'outs_when_up', 'at_bat_number', 'pitch_number', 'delta_run_exp']].copy()
    newpredictiondatapitcher = pitcher_data.copy()

    #only include pitch_type's that there are at least 100 of
    newpredictiondatapitcher =
newpredictiondatapitcher[newpredictiondatapitcher.groupby('pitch_type').pitch_type.t
ransform('count')>100].copy()

    ### Key piece- create new column that is a sum of delta_run_exp based on what
has been done in matching situations
    ### This is to create an expected run variance for a pitch situation
    groupbypitcher =
newpredictiondatapitcher.groupby(['zone','pitch_type','balls','strikes'],as_index=
False).delta_run_exp.sum()
    sorted_pitcher_data = pd.merge(newpredictiondatapitcher,groupbypitcher,
how='left', on=['zone','pitch_type','balls','strikes'])
```

```
    return sorted_pitcher_data


# mimics get_pitcher_data
def get_batter_data(batter_id):

    statcast_data_batter = statcast_batter('2018-07-01', '2022-10-31', player_id =
batter_id)
    batter_data = statcast_data_batter[['batter','description','zone','pitch_type',
'stand', 'p_throws',
'on_3b','on_2b','on_1b','outs_when_up','inning','balls','strikes','at_bat_number',
'pitch_number','delta_run_exp']].copy()
    newpredictiondatabatter = batter_data.copy()
    groupbybatter =
newpredictiondatabatter.groupby(['zone','pitch_type','balls','strikes'],as_index=
False).delta_run_exp.sum()
    sorted_batter_data = pd.merge(newpredictiondatabatter,groupbybatter, how='left',
on=['zone','pitch_type','balls','strikes'])

    return sorted_batter_data


#most data cleaning in this function
def data_prep(sorted_pitcher_data, sorted_batter_data):


    # unique = sorted_pitcher_data.pitch_type.unique()

    #the next four lines take the most common listed hand thrown with for a pitcher
and batting stance for a
    #batter, then compare them to only keep data for the correct throwing hand/
batter stance matchup
    most_common_p_throws_pitcher = sorted_pitcher_data['p_throws'].mode().iloc[0]
    sorted_batter_data = sorted_batter_data[sorted_batter_data['p_throws'] ==
most_common_p_throws_pitcher]
    most_common_stand_batter = sorted_batter_data['stand'].mode().iloc[0]
    sorted_pitcher_data = sorted_pitcher_data[sorted_pitcher_data['stand'] ==
most_common_stand_batter]

    #drop pitch_type values that are null
    sorted_pitcher_data = sorted_pitcher_data.dropna(subset=['pitch_type'])
    sorted_batter_data = sorted_batter_data.dropna(subset=['pitch_type'])

    # As seen in data exploration, batters face more pitches than most pitchers
throw. We only care about values where
    # they're seen in both pitcher and batter data as we'll never suggest a pitch
that a pitcher doesn't throw
    i = set(sorted_pitcher_data['pitch_type']) &
set(sorted_batter_data['pitch_type'])
```

```
    combined_data =
pd.concat([sorted_pitcher_data[sorted_pitcher_data['pitch_type'].isin(i)],
sorted_batter_data[sorted_batter_data['pitch_type'].isin(i)]]).sort_values('pitch_ty
pe')




    #replace null values with 0, reset the index
    combined_data.replace(np.nan,0)
    combined_data = combined_data.fillna(0)
    combined_data = combined_data.reset_index(drop=True)

    #drop columns that aren't needed anymore
    combined_data.drop(columns=['description', 'p_throws','stand'], inplace=True)

    #we will need pitch_type to be numerical
    combined_data.pitch_type = pd.Categorical(combined_data.pitch_type)
    combined_data['pitch_code'] = combined_data.pitch_type.cat.codes

    combined_data['previous_pitch'] = None
    for index, row in combined_data.iterrows():
        if row['pitch_number'] > 1:
            previous_row = combined_data.loc[
                (combined_data['at_bat_number'] == row['at_bat_number']) &
                (combined_data['pitch_number'] == row['pitch_number'] - 1)
            ]
            if not previous_row.empty:
                combined_data.at[index, 'previous_pitch'] =
previous_row['pitch_code'].iloc[0]

    combined_data['previous_pitch'] = combined_data['previous_pitch'].fillna(9)
    print(combined_data['previous_pitch'].value_counts())
    #map pitch_type so that when the numerical value is used later, we can call
pitch_type
    pitch_mapping = dict(zip(combined_data['pitch_code'],
combined_data['pitch_type']))

    'at_bat_number', 'pitch_number',

    #currently these columns have player ids in them if a player is on base, I just
want 1 or 0
    combined_data['on_1b'] = combined_data['on_1b'].where(combined_data['on_1b'] <=
1, 1)
    combined_data['on_2b'] = combined_data['on_2b'].where(combined_data['on_2b'] <=
1, 1)
    combined_data['on_3b'] = combined_data['on_3b'].where(combined_data['on_3b'] <=
1, 1)

    #drop pitch_type
```

```
    combined_data.drop(columns=['pitch_type'], inplace=True)

    #convert values to integers
    combined_data['batter'] = combined_data['batter'].astype(int)
    combined_data['zone'] = combined_data['zone'].astype(int)
    combined_data['on_3b'] = combined_data['on_3b'].astype(int)
    combined_data['on_2b'] = combined_data['on_2b'].astype(int)
    combined_data['on_1b'] = combined_data['on_1b'].astype(int)
    combined_data['inning'] = combined_data['inning'].astype(int)
    combined_data['pitcher'] = combined_data['pitcher'].astype(int)

    ### Multiply by 1000 because I need integers and delta_run_exp is a decimal
    combined_data = combined_data.multiply(1000)

    #now convert these to int
    combined_data['delta_run_exp_x'] = combined_data['delta_run_exp_x'].astype(int)
    combined_data['delta_run_exp_y'] = combined_data['delta_run_exp_y'].astype(int)

    return combined_data, pitch_mapping


#defining a function to encode
def data_preprocessing(combined_data):

    #label encoding values that are categorical
    label_encoder = LabelEncoder()
    #ordinal encoding values that have an order
    ordinal_encoder = OrdinalEncoder()
    combined_data['pitch_type_encoded'] =
label_encoder.fit_transform(combined_data['pitch_code'])
    combined_data['previous_pitch_encoded'] =
label_encoder.fit_transform(combined_data['previous_pitch'])
    combined_data['zone_encoded'] =
label_encoder.fit_transform(combined_data['zone'])
    combined_data['on_3b_encoded'] =
label_encoder.fit_transform(combined_data['on_3b'])
    combined_data['on_2b_encoded'] =
label_encoder.fit_transform(combined_data['on_2b'])
    combined_data['on_1b_encoded'] =
label_encoder.fit_transform(combined_data['on_1b'])
    combined_data['inning_encoded'] =
ordinal_encoder.fit_transform(combined_data[['inning']])
    combined_data['balls_encoded'] =
ordinal_encoder.fit_transform(combined_data[['balls']])
    combined_data['strikes_encoded'] =
ordinal_encoder.fit_transform(combined_data[['strikes']])
    combined_data['outs_encoded'] =
ordinal_encoder.fit_transform(combined_data[['outs_when_up']])

    #defining features to be used later and the ultimate target
```

```
    features = ['on_3b_encoded',
'on_2b_encoded','on_1b_encoded','inning_encoded','outs_encoded', 'balls_encoded',
'strikes_encoded', 'zone_encoded', 'pitch_type_encoded','previous_pitch_encoded']
    target = 'delta_run_exp_y'

    return features, target, combined_data, ordinal_encoder


#training model through randomforestregressor
def model_train(features, target, combined_data):

    #defining X, y and splitting data into training and test data
    X = combined_data[features]
    y = combined_data[target]
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

    #training the model
    model = RandomForestRegressor()
    model.fit(X_train, y_train)

    #model score is essentially r^2
    print(model.score(X_test,y_test))


    return features, target, model, y_test, X_test




# Predicting the Zone and Pitch Type for the Best delta_run_exp #batter_id,
pitcher_id,model,
def get_best_zone_and_pitch_type(model, features, ordinal_encoder, combined_data,
balls_encoded, strikes_encoded, on_3b_encoded, on_2b_encoded, on_1b_encoded,
inning_encoded, outs_encoded, y_test, X_test, pitch_mapping):
    # Create a dataframe with all possible combinations of zone and pitch_type

    #need number of pitches to make possible_combinations variable
    count = combined_data.pitch_type_encoded.nunique()

    #possible combinations is created to denote the number of values and particular
situation for each prediction
    possible_combinations = pd.DataFrame({

        'pitch_type_encoded': np.tile(np.arange(0, count), 13),
        'balls_encoded': [balls_encoded] * 13 * count,
        'strikes_encoded': [strikes_encoded] * 13 * count,
        'zone_encoded': np.repeat(np.arange(1, 14), count),
        'on_3b_encoded': [on_3b_encoded] * 13 * count,
        'on_2b_encoded': [on_2b_encoded] * 13 * count,
```

```
        'on_1b_encoded': [on_1b_encoded] * 13 * count,
        'inning_encoded': [inning_encoded] * 13 * count,
        'outs_encoded': [outs_encoded] * 13 * count,
        'previous_pitch_encoded': np.tile(np.arange(0, count), 13)
    })

    #Running each model for the possible combinations
    predictions = model.predict(possible_combinations[features])


    #making y_pred values for each model
    y_pred = model.predict(X_test[features])


    #divide each by 1000 to bring them back down to real numbers
    y_test = y_test.div(1000)
    y_pred = np.divide(y_pred, 1000)


    #each of these sets are going to be used to help evaluate the accuracy of the
regression models
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, y_pred)

    print(f"Mean Squared Error: {mse:.2f}")
    print(f"Root Mean Squared Error: {rmse:.2f}")
    print(f"Mean Absolute Error: {mae:.2f}")

    #get the best index by finding the minimum value of delta_run_exp_y
    best_idx = np.argmin(predictions)

    #get the best zone and pitch_type based on the index above
    best_zone = possible_combinations.loc[best_idx, 'zone_encoded']
    best_pitch_type_encoded = possible_combinations.loc[best_idx,
'pitch_type_encoded']

    #just for plot
    y_actual = y_test

    #scatter plot showing visually how well it has been predicted
    plt.figure(figsize=(8, 6))
    plt.scatter(y_actual, y_pred, color='blue', alpha=0.5)
    plt.title('Actual vs. Predicted Values')
    plt.xlabel('Actual Values')
    plt.ylabel('Predicted Values')
    plt.grid(True)
    plt.savefig('static/my_prediction_plot.png')
```

```
    #get best by inverse_transforming the pitch type and then dividing by 1000,
return pitch name also
    best_pitch_type =
ordinal_encoder.inverse_transform(best_pitch_type_encoded.reshape(-1, 1))[0,0]
    best_pitch_type = int(best_pitch_type/1000)
    pitch_code = best_pitch_type
    pitch_type_associated = pitch_mapping[pitch_code]

    return predictions, best_zone, best_pitch_type, possible_combinations,
pitch_type_associated


#main function that will take in form data on web app and then go through all the
defined functions
#can easily remove models, but for testing, I will leave them
def main(pitcher_name, batter_name, balls_encoded, strikes_encoded, outs_encoded,
on_3b_encoded, on_2b_encoded, on_1b_encoded, inning_encoded):
    pitcher_id, batter_id = get_player_id(pitcher_name, batter_name)
    sorted_pitcher_data = get_pitcher_data(pitcher_id)
    sorted_batter_data = get_batter_data(batter_id)
    combined_data, pitch_mapping = data_prep(sorted_pitcher_data,
sorted_batter_data)
    features, target, combined_data, ordinal_encoder =
data_preprocessing(combined_data)
    features, target, model, y_test, X_test = model_train(features, target,
combined_data)
    predictions, best_zone, best_pitch_type, possible_combinations,
pitch_type_associated = get_best_zone_and_pitch_type(model,  features,
ordinal_encoder, combined_data, balls_encoded, strikes_encoded, on_3b_encoded,
on_2b_encoded, on_1b_encoded, inning_encoded, outs_encoded, y_test, X_test,
pitch_mapping)
    return predictions, best_zone, best_pitch_type, possible_combinations,
pitch_type_associated


'''
pitcher_name = 'clayton kershaw'
batter_name = 'hunter pence'
balls_encoded = 0
strikes_encoded = 0
inning_encoded = 2
outs_encoded = 1
on_3b_encoded = 0
on_2b_encoded = 0
on_1b_encoded = 1

predictions, best_zone, best_pitch_type, possible_combinations,
pitch_type_associated = main(pitcher_name, batter_name, balls_encoded,
strikes_encoded, outs_encoded, on_3b_encoded, on_2b_encoded, on_1b_encoded,
inning_encoded)
```

```
#predictions, best_zone, best_pitch_type, possible_combinations =
get_best_zone_and_pitch_type( balls_encoded, strikes_encoded, on_3b_encoded,
on_2b_encoded, on_1b_encoded, inning_encoded, outs_encoded )

print(f"The best zone for the pitcher is: {best_zone}")
print(f"The best pitch type for the pitcher is: {best_pitch_type}
{pitch_type_associated}")
'''
```